Docker Compose - Complete Notes

What is Docker Compose?

Docker Compose is a tool used to define and run multi-container Docker applications using a single docker-compose.yml (or compose.yaml) file.

Why use Docker Compose?

• Manage multiple containers together.
• One command to start or stop the entire application.
• Automatically creates a network.
• Easy volume, port, and environment management.
• Ideal for development and testing.

Important Terms

Service: A container definition.
Image: Blueprint used to create containers.
Container: Running instance of an image.
Volume: Persistent storage.
Network: Communication channel between containers.
Environment Variables: Configuration values passed to containers.
Bind Mount: Maps host directory to container.
Named Volume: Docker-managed persistent storage.
depends_on: Controls startup order.
restart: Restart policy.

Basic compose.yaml Structure

version removed in modern Compose.
services:
app:
image: nginx
ports:
- "80:80"
volumes:
networks:

Common Commands

docker compose up – Create and start containers.
docker compose up -d – Run in background.
docker compose down – Stop and remove containers.
docker compose start – Start existing containers.
docker compose stop – Stop containers.
docker compose restart – Restart services.
docker compose ps – List containers.
docker compose logs – View logs.
docker compose logs -f – Follow logs.
docker compose build – Build images.
docker compose pull – Download images.
docker compose images – List images.
docker compose exec SERVICE bash – Open shell.
docker compose run SERVICE CMD – Run one-time command.

docker compose config – Validate configuration.
docker compose top – Running processes.
docker compose events – Live events.
docker compose kill – Force stop.
docker compose rm – Remove stopped containers.
docker compose down --volumes – Remove volumes.
docker compose down --rmi all – Remove images.

Compose File Keywords

services: Define containers.
image: Image name.
build: Build using Dockerfile.
container_name: Custom container name.
ports: Host:Container ports.
volumes: Storage mapping.
environment: Environment variables.
env_file: Load variables from file.
depends_on: Service dependency.
command: Override default command.
working_dir: Working directory.
restart: Restart policy.
networks: Attach network.
healthcheck: Container health test.

Example

services:
web:
build: .
ports:
- "5000:5000"
volumes:
- .:/app
depends_on:
- db
db:
image: postgres:16
environment:
POSTGRES_PASSWORD: secret

Workflow

1. Write Dockerfile.
2. Create compose.yaml.
3. Run **docker compose up -d**.
4. Check with **docker compose ps**.
5. View logs.
6. Stop using **docker compose down**.

Interview Points

• Compose manages multiple containers.
• Default network is created automatically.
• Service names become DNS hostnames.
• Volumes preserve data.

- Best for development and testing.
- Kubernetes is preferred for large-scale production.